

UNIT 2

Introduction to Feature Extraction

2

Introduction to Feature Extraction:

One-hot encoding methods, BoW, skipgrams, CountVectorizer, TF-IDF. Probabilistic language modeling, Nave Bayes and its applications. Markov models, N-grams, Estimating the probability of a word, and smoothing. Generative models of language.

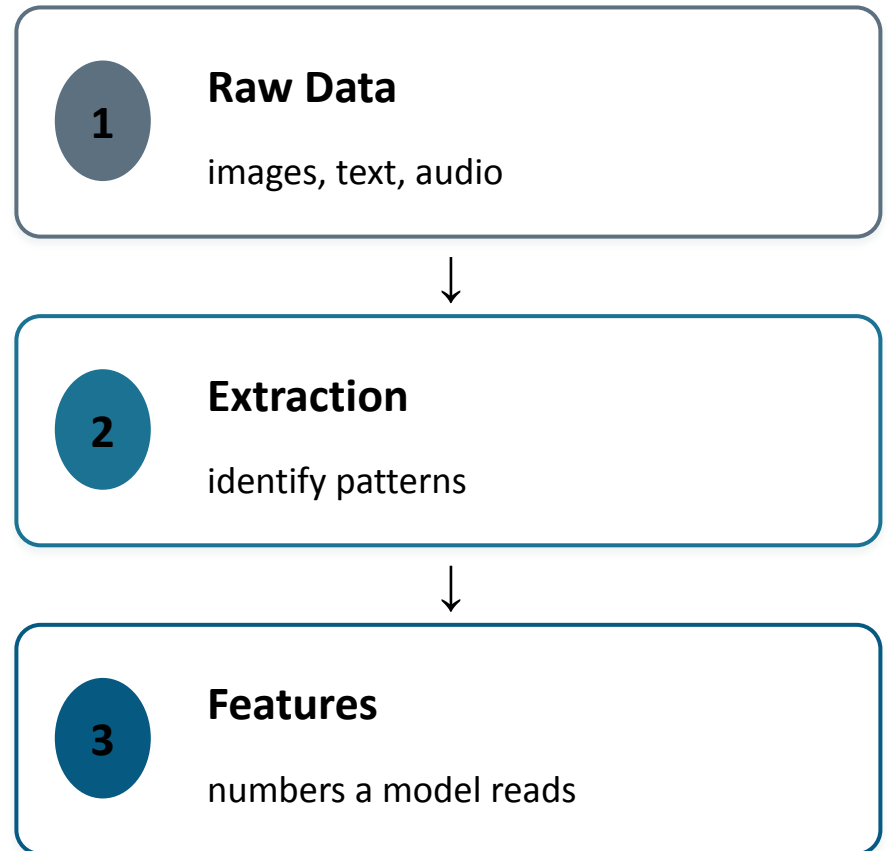
Feature Extraction

What Is Feature Extraction?

Feature extraction means taking raw data and converting it into important, useful pieces of information (features) that a machine learning model can understand.

Feature extraction = converting large, raw data into smaller and meaningful information for a machine learning model.

The Transformation



Why Is Feature Extraction Useful?

Reduces Data

Strips out unnecessary and redundant information, keeping only what matters.

Makes Processing Faster

Models work with smaller, denser data — so training and inference speed up.

Improves Learning

Helps the model focus on the important patterns instead of raw noise.

Examples of Extracted Features

Image

Edges, shapes, colors

Text

Words, keywords, word frequency

Audio

Pitch, tone, frequency

Feature Extraction in NLP

Example

Suppose we have a customer review:

“The mobile phone has excellent camera quality.”

A machine cannot directly process this sentence as text.

Feature extraction converts it into numerical information such as:

- **Words** → mobile, phone, excellent, camera, quality
- **Word frequency**
- **Importance of each word**
- **Numerical vectors representing the words**

So:

Text → Feature Extraction → Numerical Representation → Machine Learning Model

Also known as:

Text Representation

Text Extraction

Text Vectorization

Important Aspects Related to NLP Feature Extraction

What is feature extraction from text?

The process of converting text data into numbers is called Feature Extraction from the text.

Why do we need it?

Machines can only understand numbers and to make machines able to identify language, we need to convert it into numeric form.

Why is it difficult?

Unlike images, which are already represented as pixels (numbers), and audio, which is represented as waveform signals, text has meaning, context, and ambiguity. Therefore, representing a sentence numerically while preserving its meaning is challenging.

1. One-Hot Encoding
2. Bag of Words (BoW)
3. N-grams
4. TF-IDF
5. Custom Features
6. Word Embeddings such as Word2Vec

What are the techniques?

Common Terms Used in NLP

Corpus

A collection of text documents used for NLP analysis.

Example 1,000 Amazon product reviews together form the corpus.

Vocabulary

The collection of unique words present in the corpus.

Example "I like Python." + "I like NLP." → {I, like, Python, NLP} — size 4.

Document

One individual text record within a corpus.

Example "This phone has a good camera." is a single document.

This single review is one **document**.

If we have 1,000 reviews:

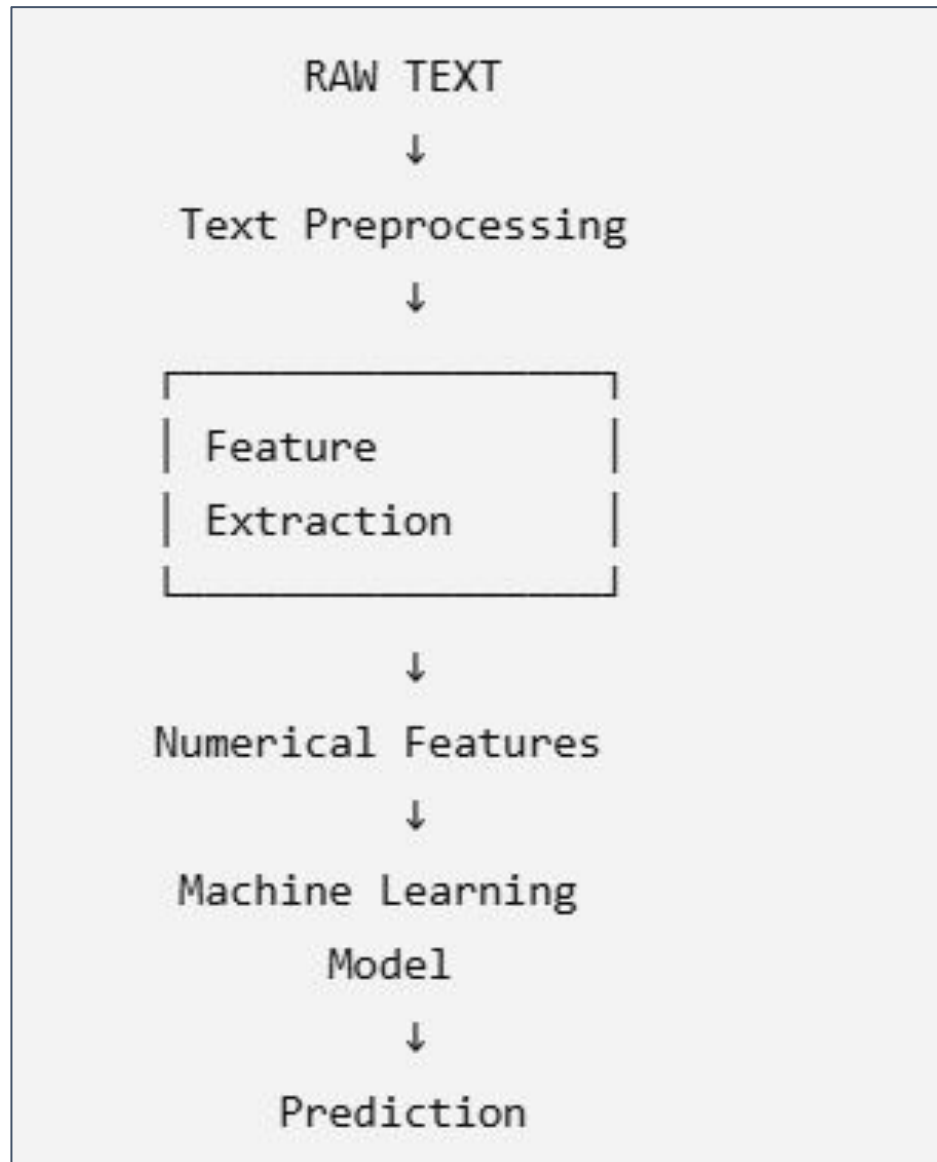
Corpus = 1,000 documents

Word

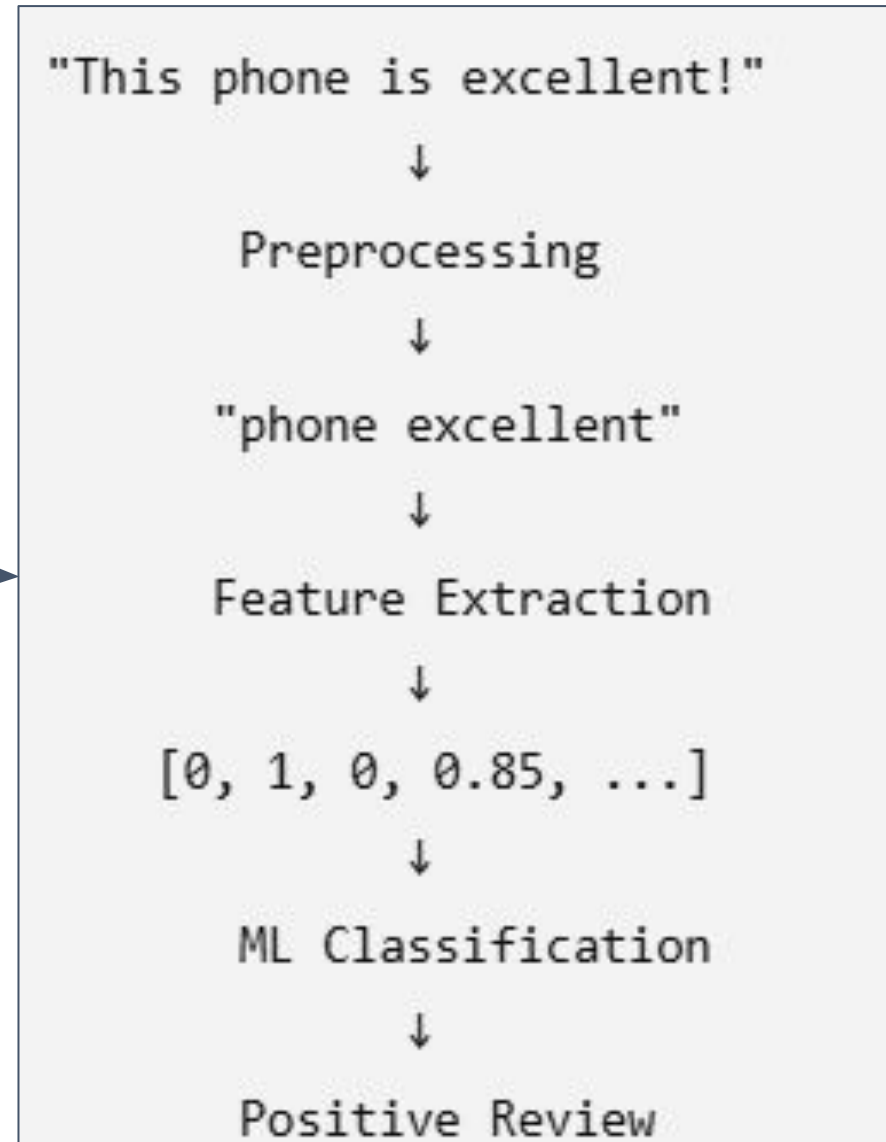
An individual linguistic unit appearing in a document.

Example "This phone is excellent." → This | phone | is | excellent.

Feature Extraction



Example



Feature Extraction Methods

One-Hot Encoding

One-Hot Encoding means representing each word using a vector containing 0s and a single 1. The position of 1 tells us which word it represents

Example: One-Hot Encoding

Suppose we have three simple sentences:

- Document 1: I like pizza
- Document 2: I like burger
- Document 3: I eat pizza

These are the documents

One-Hot Encoding, Step by Step

STEP 1 • Corpus

Doc 1 — "I like pizza"

Doc 2 — "I like burger"

Doc 3 — "I eat pizza"

STEP 2 • Vocabulary (V = 5)



STEP 3 • One-Hot Representation

Word	I	like	pizza	burger	eat
I	1	0	0	0	0
like	0	1	0	0	0
pizza	0	0	1	0	0
burger	0	0	0	1	0
eat	0	0	0	0	1

Document 1 "I like pizza" →

I → [1,0,0,0,0]

like → [0,1,0,0,0]

pizza → [0,0,1,0,0]

Repeated Word in One-Hot Encoding

Corpus:

“I like Python”

“Python is easy”

“I like Python”

Vocabulary: I, like, Python, is, easy → 5 unique words

Python → [0, 0, 1, 0, 0]

Even when Python appears multiple times, its vector remains the same.

One word = one fixed position = one fixed One-Hot vector.

One-Hot Encoding

Advantages

1. **Simple and intuitive** – Easy to understand.
2. **Easy to implement** – Straightforward to apply to text data.

Disadvantages

1. **Creates sparsity** – Most values in the vector are 0.
2. **Different document lengths** – Different texts can produce vectors of different sizes, which ML models cannot directly handle.
3. **OOV (Out-of-Vocabulary) problem** – A new word appearing during prediction may not exist in the vocabulary.
4. **No semantic meaning** – It treats every word independently and cannot capture relationships between words.

Therefore: One-hot encoding is useful for understanding the basic concept, but it is generally not preferred for large-scale NLP applications because of these limitations.

Question : One Hot Encoding

* Consider the following small corpus :

- ① "I like Python"
- ② "I like NLP"
- ③ "Python is easy"

Perform One Hot Encoding for the words in the corpus.

Tasks :

1. Identify the unique vocabulary.
2. Assign a position to each word.
3. Represent each word using a One Hot vector.
4. Write the One Hot vector for "Python", "NLP" and "easy".

Answer Format (Example) :

Vocabulary = [I , like , Python , NLP , is , easy]

One Hot Vectors :

Word	I	like	Python	NLP	is	easy
I	1	0	0	0	0	0
like	0	1	0	0	0	0
Python	0	0	1	0	0	0
NLP	0	0	0	1	0	0
is	0	0	0	0	1	0
easy	0	0	0	0	0	1

For example ,

Python $\rightarrow$ [0 , 0 , 1 , 0 , 0 , 0]

NLP $\rightarrow$ [0 , 0 , 0 , 1 , 0 , 0]

easy $\rightarrow$ [0 , 0 , 0 , 0 , 0 , 1]

Note : Here '1' shows the position of the word
and all other positions are '0'.

One Hot Encoding

* One hot encoding is a way to represent categorical data as binary vectors.

→ Only one element is '1' (hot) and the rest are '0' (cold).

Example:

Colors = [Red , Green , Blue]

Step 1: List all categories (vocabulary)

Red , Green , Blue

Step 2: Create binary vector for each category

Color	Red	Green	Blue
Red	1	0	0
Green	0	1	0
Blue	0	0	1

So,

Red → [1 , 0 , 0]
Green → [0 , 1 , 0]
Blue → [0 , 0 , 1]

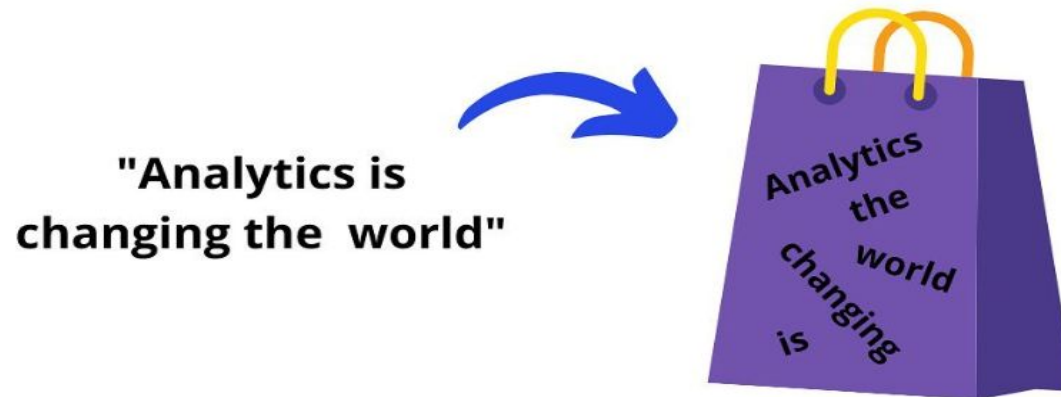
} This is One Hot
Encoding.

Bag-of-Words (BoW)

What is Bag of Words?

- Bag of Words (BoW) is a simple technique used in NLP **to convert text into numbers**.
- It creates a list of all unique words in the given documents and then counts how many times each word appears in each document.

Note: BoW considers the words and their frequency, but it does not consider the order of words.



Bag-of-Words (BoW) Example: The Three Reviews

1

Review 1

"The laptop has fast processor and fast performance"

2

Review 2

"The laptop has good battery and good performance"

3

Review 3

"The processor is fast and the battery is good"

Step 1: Build the Vocabulary

Collect every unique word that appears across all three reviews.

the

laptop

has

fast

processor

and

performance

good

battery

is

VOCABULARY SIZE

10

Step 2 · Count Word Frequency

For each review, count how many times each vocabulary word occurs (focuses on word frequency)

	the	laptop	has	fast	processor	and	performance	good	battery	is
Review 1	1	1	1	2	1	1	1	0	0	0
Review 2	1	1	1	0	0	1	1	2	1	0
Review 3	2	0	0	1	1	1	0	1	1	1

"fast" appears twice in Review 1 → count = 2. "good" appears twice in Review 2 → count = 2.

Review 1
"The laptop has <u>fast</u> processor and <u>fast</u> performance"
Review 2
"The laptop has <u>good</u> battery and <u>good</u> performance"
Review 3
"The processor is <u>fast</u> and the battery is <u>good</u> "

Step 3 · Convert into Numerical Vectors

Each review becomes a 10-dimensional vector of word counts.

Review 1

The laptop has fast processor and fast performance

fast = 2 (appears twice)

[1, 1, 1, 2, 1, 1, 1, 0, 0, 0]

Review 2

The laptop has good battery and good performance

good = 2 (appears twice)

[1, 1, 1, 0, 0, 1, 1, 2, 1, 0]

Review 3

The processor is fast and the battery is good

the = 2 (appears twice)

[2, 0, 0, 1, 1, 1, 0, 1, 1, 1]

Bag of Words vs. One-Hot Encoding

Feature	One-Hot Encoding	Bag of Words
Represents	Individual word	Entire document
Values	0 or 1	Word frequency
Repeated word	Still 1	Can become 2, 3, 4...
Example	good → [0,0,1,0]	good good → good = 2
Captures frequency?	No	Yes
Understands meaning?	No	No

An Important Limitation of BoW

Bag of Words does not understand the meaning of words — only their counts.

"The processor is fast"

processor → 1
fast → 1

"The processor is not fast"

processor → 1
fast → 1
not → 1

These two sentences produce nearly identical vectors — even though their meaning is opposite.

Bag-of-Words (BoW) – Advantages and Disadvantages

Advantages	Disadvantages
Simple	Ignores word order
Easy to implement	Ignores grammar
Fast	Produces sparse vectors
Captures word frequency	Large vocabulary
Good for basic classification	Cannot understand meaning
Works well on small datasets	Problems with unseen words

The Core Problem in BoW

Bag of Words ignores the order of words in a sentence.

It looks at which words are present and how many times they occur — not how they relate to each other or their position.

Consider these two sentences:

"This is a good job."

"This is not a good job."

BoW counts the words but doesn't capture that “not good” changes positive → negative.

Same Words, Opposite Meaning

Both sentences use exactly the same three words: dog, bites, man.

"Dog bites man."

Dog is doing the biting.

dog → bites → man

"Man bites dog."

Man is doing the biting.

man → bites → dog

BoW can produce the same vector representation for both — but their meanings are completely different! (if order changes then meaning changes)

What Information Does BoW Lose?

BoW Limitation	What Happens
Word order	Lost
Relationship between nearby words	Lost
Context	Limited
Phrases such as “not good”	Not captured properly
Sentence structure / grammar	Not captured

S1: Students learn Python for business.

S2: Students use Python for data analysis.

S3: Business students use data for decision making.

1

Identify the vocabulary (unique words).

2

Create the Bag of Words vector for each sentence.

3

Which words are repeated across sentences?

4

What is the frequency of students, Python, and business in each sentence?

Answer 1

Vocabulary (Unique Words)

Combine all words from $S_1, S_2, S_3 \rightarrow$ lowercase $\rightarrow$ remove duplicates.

students

learn

python

for

business

use

data

analysis

decision

making

Total unique words = **10**

Answer 2

Bag of Words Vectors

Sentence	students	learn	python	for	business	use	data	analysis	decision	making
S1	1	1	1	1	1	0	0	0	0	0
S2	1	0	1	1	0	1	1	1	0	0
S3	1	0	0	1	1	1	1	0	1	1

S1 → [1, 1, 1, 1, 1, 0, 0, 0, 0, 0]

S2 → [1, 0, 1, 1, 0, 1, 1, 1, 0, 0]

S3 → [1, 0, 0, 1, 1, 1, 1, 0, 1, 1]

Answer 3

Words Repeated Across Sentences

students

appears in S1, S2, S3 (all three)

for

appears in S1, S2, S3 (all three)

python

appears in S1, S2

business

appears in S1, S3

use

appears in S2, S3

data

appears in S2, S3

Answer 4

Frequency: students, Python, business

Word	S1	S2	S3
students	1	1	1
Python	1	1	0
business	1	0	1

Key Takeaway

Bag of Words represents text as a vector of word counts.

It ignores grammar and word order — only which words appear, and how many times, matters.

This Is Where N-Grams Help

Bag-of-N-Grams Model?

A Bag-of-N-Grams represents a sentence/document **using groups of N consecutive words**, instead of considering only individual words.

gram = a group/sequence of words or tokens

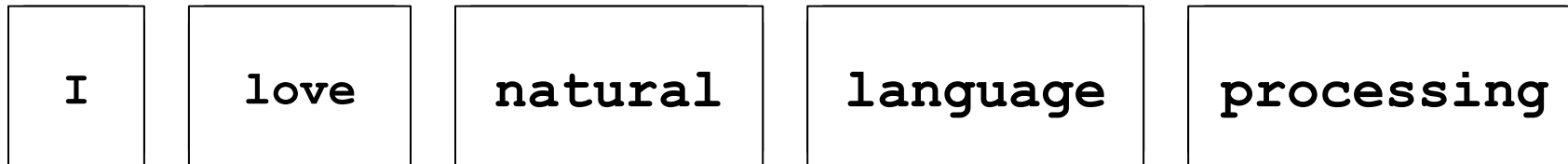
N-GRAMS

Types: Unigrams, Bigrams, Trigrams & Beyond

The Example Sentence

We'll break this 5-word sentence into N-Grams of different sizes.

"I love natural language processing."



Total words in sentence: **N = 5**

N = 1

Unigram — 1 Word at a Time

Here, $N = 1$, so we take one word at a time.

I

love

natural

language

processing

Bag of Unigrams

[I, love, natural, language, processing]

Unigrams are exactly what plain Bag of Words uses — single words, no order.

$N = 2$

Bigram — 2 Words at a Time

Here, $N = 2$, so we take two consecutive words, sliding one word at a time.

I love

love natural

natural language

language
processing

Bag of Bigrams

[I love, love natural, natural language, language processing]

Notice each bigram contains 2 words, and consecutive bigrams overlap by one word.

N = 3

Trigram — 3 Words at a Time

Here, $N = 3$, so we take three consecutive words.

I love natural

love natural language

natural language processing

Bag of Trigrams: [I love natural, love natural language, natural language processing]

The General Formula

If a sentence has N words and we build an n -gram, the count of n -grams is:

$$N - n + 1$$

N = total words in the sentence

n = size of the n -gram (1, 2, 3, ...)

Our sentence has 5 words, so $N = 5$ throughout the examples

Formula Applied

Using N = 5 words, here's how many n-grams each size produces.

N-Gram Type	n	Calculation	Count
Bigram	2	$5 - 2 + 1$	4
Trigram	3	$5 - 3 + 1$	3
4-gram	4	$5 - 4 + 1$	2

4-gram

[I love natural language, love natural language processing]

Small Difference

BoW

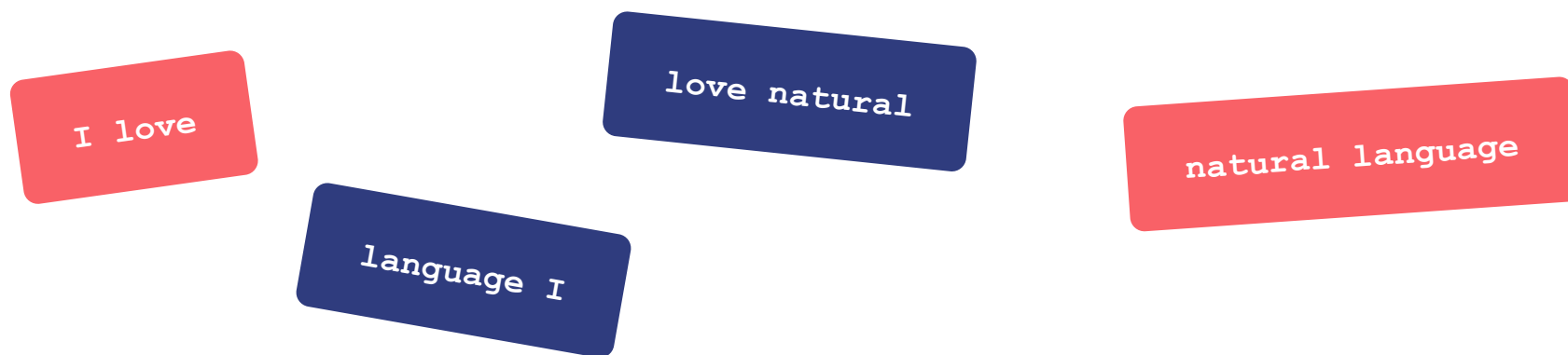
Knows which words are present.

N-Grams

Knows which words occur together and preserves some word order.

So, Why “Bag”? (Bi-gram)

Picture tossing all the n-grams into a bag.



Once inside the bag, there's no sense of first, second, or last — **just a collection of items.**

This is exactly why it's called a Bag of N-Grams.

Q. Given the sentence:

"NLP is fun and NLP is powerful"

Tasks:

- a) List all the 1-grams (unigrams) and their counts.
- b) List all the 2-grams (bigrams) and their counts.
- c) List all the 3-grams (trigrams) and their counts.
- d) What is the total number of 2-grams in the sentence?

Solution:

Step 1: Tokenize the sentence

[NLP, is, fun, and, NLP, is, powerful]

a) 1-grams (unigrams)

Unigram	Count
NLP	2
is	2
fun	1
and	1
powerful	1

b) 2-grams (bigrams)

Bigram	Count
NLP is	1
is fun	1
fun and	1
and NLP	1
NLP is	1
is powerful	1

c) 3-grams (trigrams)

Trigram	Count
NLP is fun	1
is fun and	1
fun and NLP	1
and NLP is	1
NLP is powerful	1

d) Total number of 2-grams = 6

COUNTVECTORIZER

Turning Reviews Into Numbers

What Is CountVectorizer?

CountVectorizer is a tool from Python's scikit-learn library that converts text into numbers so a machine learning model can process it.

means that it reads sentences, finds all unique words, and counts how many times each word appears in each sentence. (that is by BOW checks for word frequency)

Uses Bag of Words

which words, how often

Outputs a numerical matrix

rows = documents, columns = words

Example Student Reviews

Consider the below these three reviews

Document 1 *"Python is easy"*

Document 2 *"Python is powerful"*

Document 3 *"Python is easy and powerful"*

STEP 1

Vocabulary & Document-Term Matrix

CountVectorizer identifies all unique words — these become the columns.

Python

is

easy

powerful

and

Document	Python	is	easy	powerful	and
Document 1	1	1	1	0	0
Document 2	1	1	0	1	0
Document 3	1	1	1	1	1

This is called the Document-Term Matrix.

STEP 2

Understand Each Row

Each document becomes a vector by counting vocabulary words. (from the document matrix)

D1

"Python is easy."

[1, 1, 1, 0, 0]

D2

"Python is powerful."

[1, 1, 0, 1, 0]

D3

"Python is easy and powerful."

[1, 1, 1, 1, 1]

What About Repeated Words?

This is where CountVectorizer becomes particularly useful.

D4: "Python is easy and Python is powerful"

Python

2

is

2

easy

1

powerful

1

and

1

Vector: **[2, 2, 1, 1, 1]**

CountVectorizer doesn't just mark presence with a 1 — it counts actual occurrences.

Fit and transform the three documents (Without configured).

```
from sklearn.feature_extraction.text import CountVectorizer

documents = [
    "Python is easy",
    "Python is powerful",
    "Python is easy and powerful"
]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(documents)

print(vectorizer.get_feature_names_out())
print(X.toarray())
```

The Output

Notice the vocabulary order has changed.

```
['and' 'easy' 'is' 'powerful' 'python']  
[[0 1 1 0 1]  
 [0 0 1 1 1]  
 [1 1 1 1 1]]
```

We might expect:

**Python, is, easy, powerful,
and**

CountVectorizer actually creates:

and, easy, is, powerful, python

CountVectorizer normally arranges the vocabulary alphabetically — that's why the column order changed.

Understanding fit_transform()

Two things happen when you call `vectorizer.fit_transform(documents)`.

```
vectorizer = CountVectorizer()  
X = vectorizer.fit_transform(documents)
```

1. fit()

Learns the vocabulary from the documents.

`and · easy · is · powerful · python`

2. transform()

Converts each document into a vector of word counts.

`"Python is easy" → [0, 1, 1, 0, 1]`

fit = learn the vocabulary

transform = convert text into numerical vectors

fit_transform = do both together

- In Scikit-Learn, initializing `CountVectorizer()` without arguments actually means it uses its factory defaults:
 - `stop_words=None` - Stop words are NOT removed by default.
 - `lowercase=True` - Text is converted to lowercase.
 - `Token pattern` - Keeps all words of 2 or more alphanumeric characters (single letters like "a" are ignored).

Key Configuration Parameters

```
from sklearn.feature_extraction.text import CountVectorizer

documents = [
    "Python is easy",
    "Python is powerful",
    "Python is easy and powerful"
]

# CONFIGURED CountVectorizer
vectorizer = CountVectorizer(
    stop_words=['is', 'and'], # Custom stop words list to remove unwanted words
    ngram_range=(1, 2), # Captures single words (unigrams) AND 2-word combinations (bigrams)
    lowercase=True # Ensures case insensitivity (default is True)
)

X = vectorizer.fit_transform(documents)

print("Features Extracted:")
print(vectorizer.get_feature_names_out())
```

ngram_range=(min_n, max_n): The range of word combinations to extract. Here, (1, 2) means start at 1-word groups and go up to 2-word groups.

- Features Extracted:
['easy' 'easy powerful' 'powerful' 'python' 'python easy'
'python powerful']

```

import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer

documents = [
    "Python is easy",
    "Python is powerful",
    "Python is easy and powerful"
]

# CONFIGURED CountVectorizer
vectorizer = CountVectorizer(
    stop_words=['is', 'and'], # Custom stop words list to remove unwanted words
    ngram_range=(1,1), # Captures single words (unigrams) Single words only
    lowercase=True # Ensures case insensitivity (default is True)
)

X = vectorizer.fit_transform(documents)
# Wrap in Pandas DataFrame to easily read features alongside the matrix
df = pd.DataFrame(
    X.toarray(),
    columns=vectorizer.get_feature_names_out(),
    index=['Doc 1', 'Doc 2', 'Doc 3']
)

print("Features Extracted:")
print(vectorizer.get_feature_names_out())
print("\nDocument-Term Matrix:")
print(df)

```

- Features Extracted:
['easy' 'powerful' 'python']

Document-Term Matrix:

	easy	powerful	python
Doc 1	1	0	1
Doc 2	0	1	1
Doc 3	1	1	1

Question

Consider the following corpus:

1. “The fast fox jumps over the lazy dog.”
2. “The dog is very fast.”
3. “Lazy dogs are sleeping under the warm sun.”

Using CountVectorizer with ngram_range=(1,1), perform the following:

1. Extract the features (unigrams) from the given corpus.
2. Construct the document-term matrix by counting the occurrence of each feature in each document.

Solution

Features Extracted:

```
['are' 'dog' 'dogs' 'fast' 'fox' 'jumps' 'lazy' 'over' 'sleeping' 'sun'  
'the' 'under' 'very' 'warm']
```

Document-Term Matrix:

	are	dog	dogs	fast	fox	jumps	lazy	over	sleeping	sun	the	\
Doc 1	0	1	0	1	1	1	1	1	0	0	2	
Doc 2	0	1	0	1	0	0	0	0	0	0	1	
Doc 3	1	0	1	0	0	0	1	0	1	1	1	

	under	very	warm
Doc 1	0	0	0
Doc 2	0	1	0
Doc 3	1	0	1

TF-IDF

Term Frequency–Inverse Document Frequency

What Is TF-IDF?

TF-IDF (Term Frequency – Inverse Document Frequency) is a technique used in NLP to find out how important a word is in a document compared with a collection of documents.

Words like “the”, “is”, “very”

appear in many documents → low TF-IDF score

Words like “transformer”

frequent in one doc, rare elsewhere → high TF-IDF score

In short: TF-IDF helps us identify the important and unique words in a document.

Document vs. Corpus

Document

A single piece of text — a sentence, paragraph, review, or article.

"The AI model is very good."

Corpus

A collection of documents.

D1: The AI model is very good.

D2: The AI model is very fast.

D3: Machine learning is very useful.

Together, these three documents form a corpus.

The Two Parts of TF-IDF

TF

Term Frequency

How often a word appears in a particular document.

IDF

Inverse Document Frequency

How rare or common a word is across all documents in the corpus.

$$\text{TF-IDF} = \text{TF} \times \text{IDF}$$

High score = frequent in one document, but rare across the others.

Example

D1

"AI students learn Python"

Total words in document: 4 words

D2

"AI students learn machine learning"

Total words in document: 5 words

D3

"Python is useful for AI students"

Total words in document: 6 words

STEP 1

Tokenization

Break each document into individual words (tokens).

Document	Tokens
D1	AI, students, learn, Python
D2	AI, students, learn, machine, learning
D3	Python, is, useful, for, AI, students

STEP 2

Build the Vocabulary

Combine all words and remove duplicates.

1 AI

2 students

3 learn

4 Python

5 machine

6 learning

7 is

8 useful

9 for

9 unique words → these become our features

STEP 3**TF for Document 1**

D1 = "AI students learn Python" · Total words = 4

Word	TF
AI	$1/4 = 0.25$
students	$1/4 = 0.25$
learn	$1/4 = 0.25$
Python	$1/4 = 0.25$

Each word occurs exactly once, so every word shares the same TF.

STEP 3**TF for Document 2**

D2 = "AI students learn machine learning" · Total words = 5

Word	TF
AI	$1/5 = 0.20$
students	$1/5 = 0.20$
learn	$1/5 = 0.20$
machine	$1/5 = 0.20$
learning	$1/5 = 0.20$

Notice: “learn” and “learning” are treated as two different words.

STEP 3

TF for Document 3

D3 = "Python is useful for AI students" · Total words = 6

Word	TF
Python	$1/6 = 0.167$
is	$1/6 = 0.167$
useful	$1/6 = 0.167$
for	$1/6 = 0.167$
AI	$1/6 = 0.167$
students	$1/6 = 0.167$

STEP 4

Now we look at the entire corpus of 3 documents.

$$\text{IDF} = \log \left(N / \text{DF} \right)$$

N = total documents (3) DF = documents containing the word

AI, students

DF = 3

$$\log(3/3) = 0$$

learn, Python

DF = 2

$$\log(3/2) \approx 0.176$$

machine, learning, is,
useful, for

DF = 1

$$\log(3/1) \approx 0.477$$

D1 "AI students learn Python"

Total words in document: 4 words

D2 "AI students learn machine learning"

Total words in document: 5 words

D3 "Python is useful for AI students"

Total words in document: 6 words

IDF for All 9 Words

Word	Documents	DF	IDF
AI	D1, D2, D3	3	0
students	D1, D2, D3	3	0
learn	D1, D2	2	0.1761
Python	D1, D3	2	0.1761
machine	D2	1	0.4771
learning	D2	1	0.4771
is	D3	1	0.4771
useful	D3	1	0.4771
for	D3	1	0.4771

STEP 5**TF-IDF for Document 1**

D1 = "AI students learn Python"

Word	TF	IDF	TF-IDF
AI	0.25	0	0
students	0.25	0	0
learn	0.25	0.1761	0.0440
Python	0.25	0.1761	0.0440

D1 Vector: [0, 0, 0.0440, 0.0440]

STEP 5**TF-IDF for Document 2**

D2 = "AI students learn machine learning"

Word	TF	IDF	TF-IDF
AI	0.20	0	0
students	0.20	0	0
learn	0.20	0.1761	0.0352
machine	0.20	0.4771	0.0954
learning	0.20	0.4771	0.0954

The higher scores of “machine” and “learning” make sense — they occur only in D2.

STEP 5

TF-IDF for Document 3

D3 = "Python is useful for AI students"

Word	TF	IDF	TF-IDF
Python	0.167	0.1761	0.0294
is	0.167	0.4771	0.0795
useful	0.167	0.4771	0.0795
for	0.167	0.4771	0.0795
AI	0.167	0	0
students	0.167	0	0

Final TF-IDF Matrix

Vocabulary: AI, students, learn, Python, machine, learning, is, useful, for

Doc	AI	students	learn	Python	machine	learning	is	useful	for
D1	0	0	0.0440	0.0440	0	0	0	0	0
D2	0	0	0.0352	0	0.0954	0.0954	0	0	0
D3	0	0	0	0.0294	0	0	0.0795	0.0795	0.0795

Each row is now a numeric vector representing its document — ready for machine learning.

Why Do We Take a Log in IDF?

Log controls the IDF value so a very rare word doesn't get an extremely large weight.

Example: 1,000 documents. "transformer" appears in only 1 document (DF = 1). suppose TF = 0.2 (within range (0 and 1))

WITHOUT LOG

$$\text{IDF} = N / \text{DF}$$

$$\text{IDF} = 1000 / 1 = 1000$$

$$\text{TF-IDF} = 0.2 \times 1000 = 200$$

IDF dominates completely — the score explodes.

WITH LOG

$$\text{IDF} = \log(N / \text{DF})$$

$$\text{IDF} = \log(1000) \approx 3$$

$$\text{TF-IDF} = 0.2 \times 3 = 0.6$$

Much more balanced — stays in a reasonable range.

Given Documents

Document 1 : AI students learn Python

Document 2 : AI students learn machine learning

Document 3 : Python is useful for AI students

Step 1: Tokenization

Document	Tokens
D1	AI, students, learn, Python
D2	AI, students, learn, machine, learning
D3	Python, is, useful, for, AI, students

$$TF = \frac{\text{No. of times a word appears}}{\text{Total number of words in the document}}$$

$$IDF = \log \left(\frac{N}{DF} \right)$$

N = total no. of documents
 DF = no. of documents containing the word

Step 2: Unique Vocabulary

AI, students, learn, Python,
machine, learning, is, useful, for
(Total 9 unique words)

Step 3: Term Frequency (TF)

D1 : AI students learn Python
(Total words = 4)

Word	TF
AI	$1/4 = 0.25$
students	$1/4 = 0.25$
learn	$1/4 = 0.25$
Python	$1/4 = 0.25$

D2 : AI students learn machine learning
(Total words = 5)

Word	TF
AI	$1/5 = 0.20$
students	$1/5 = 0.20$
learn	$1/5 = 0.20$
machine	$1/5 = 0.20$
learning	$1/5 = 0.20$

D3 : Python is useful for AI student
(Total words = 6)

Word	TF
Python	$1/6 = 0.167$
is	$1/6 = 0.167$
useful	$1/6 = 0.167$
for	$1/6 = 0.167$
AI	$1/6 = 0.167$
students	$1/6 = 0.167$

Step 4: Inverse Document Frequency (IDF)

Word	Documents containing word	DF	$IDF = \log \left(\frac{N}{DF} \right)$	IDF value
AI	D1, D2, D3	3	$\log (3/3) = \log (1)$	0
students	D1, D2, D3	3	$\log (3/3) = \log (1)$	0
learn	D1, D2	2	$\log (3/2)$	0.1761
Python	D1, D3	2	$\log (3/2)$	0.1761
machine	D2	1	$\log (3/1)$	0.4771
learning	D2	1	$\log (3/1)$	0.4771
is	D3	1	$\log (3/1)$	0.4771
useful	D3	1	$\log (3/1)$	0.4771
for	D3	1	$\log (3/1)$	0.4771

Step 5: TF-IDF = TF × IDF

D1: AI students learn Python
(Total words = 4)

Word	TF	IDF	TF-IDF
AI	0.25	0	0
students	0.25	0	0
learn	0.25	0.1761	0.0440
Python	0.25	0.1761	0.0440

D1 vector → [0, 0, 0.0440, 0.0440,
0, 0, 0, 0, 0]

D2: AI students learn machine learning
(Total words = 5)

Word	TF	IDF	TF-IDF
AI	0.20	0	0
students	0.20	0	0
learn	0.20	0.1761	0.0352
machine	0.20	0.4771	0.0954
learning	0.20	0.4771	0.0954

D2 vector → [0, 0, 0.0352, 0,
0.0954, 0.0954, 0, 0, 0]

D3: Python is useful for AI students
(Total words = 6)

Word	TF	IDF	TF-IDF
Python	0.167	0.1761	0.0294
is	0.167	0.4771	0.0795
useful	0.167	0.4771	0.0795
for	0.167	0.4771	0.0795
AI	0.167	0	0
students	0.167	0	0

D3 vector → [0, 0, 0, 0.0294, 0,
0, 0.0795, 0.0795, 0.0795]

Final TF-IDF Matrix

Document	AI	students	learn	Python	machine	learning	is	useful	for
D1	0	0	0.0440	0.0440	0	0	0	0	0
D2	0	0	0.0352	0	0.0954	0.0954	0	0	0
D3	0	0	0	0.0294	0	0	0.0795	0.0795	0.0795

Question (TF-IDF)

Sent 1 : good boy

Sent 2 : good girl

Sent 3 : boy girl good

$$TF = \frac{\text{No. of rep of words in a sentence}}{\text{No. of words in a sentence}}$$

$$IDF = \log \left(\frac{\text{No. of sentences}}{\text{No. of sentences containing word}} \right)$$

Solution – (TF-IDF)

TF (Term Frequency)

Word	S1	S2	S3
good	1/2	1/2	1/3
boy	1/2	0	1/3
girl	0	1/2	1/3

IDF (Inverse Document Frequency)

Word	IDF = $\log \left(\frac{\text{No. of sentences}}{\text{No. of sentences containing word}} \right)$
good	$\log \left(\frac{3}{3} \right) = \log (1) = 0$
boy	$\log \left(\frac{3}{2} \right) = 0.1761$
girl	$\log \left(\frac{3}{2} \right) = 0.1761$

TF-IDF = TF × IDF

Word	good	boy	girl
Sent 1	$0.5 \times 0 = 0$	$0.5 \times 0.1761 = 0.0880$	$0 \times 0.1761 = 0$
Sent 2	$0.5 \times 0 = 0$	$0 \times 0.1761 = 0$	$0.5 \times 0.1761 = 0.0880$
Sent 3	$(1/3) \times 0 = 0$	$(1/3) \times 0.1761 = 0.0587$	$(1/3) \times 0.1761 = 0.0587$

$$\frac{1}{2} \log \left(\frac{3}{2} \right) = 0.5 \times 0.1761 \\ = 0.0880$$

$$\frac{1}{3} \log \left(\frac{3}{2} \right) = 0.333 \times 0.1761 \\ = 0.0587$$

Word Importance in TF-IDF — A Big Improvement over BoW

Unlike Bag of Words (BoW), TF-IDF does not treat all words equally.

- Common words get a lower weight because they appear in many documents.
- Important or rare words get a higher weight because they help identify the document.
- For example, words like “good” may appear in many documents, while words like “boy” or “girl” may be more useful in a particular context.

Therefore, TF-IDF automatically gives more importance to meaningful words.

This is one of the major advantages of TF-IDF over BoW.

Disadvantages of TF-IDF

1. **Sparsity:** Most values are zero → requires more memory and computation.
2. **OOV Problem:** Cannot represent new or unseen words that are not in the vocabulary.(As TF IDF of that word is not calculated)
3. **No Meaning/Context:** It gives word importance but does not understand the meaning or relationship between words.

Therefore, we move from **TF-IDF** to **Word2Vec**, which can capture word relationships and meanings.

The Code

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
# Some example documents
```

```
documents = [  
    "I like Python",  
    "I like NLP",  
    "Python is easy"  
]
```

```
['easy' 'is' 'like' 'nlp' 'python']  
[[0.          0.          0.70710678 0.          0.70710678]  
 [0.          0.          0.60534851 0.79596054 0.          ]  
 [0.62276601 0.62276601 0.          0.          0.4736296 ]]
```

```
# Create TF-IDF vectorizer
```

```
vectorizer = TfidfVectorizer()
```

```
# Convert text into TF-IDF values
```

```
tfidf_matrix = vectorizer.fit_transform(documents)
```

```
# Display words
```

```
print(vectorizer.get_feature_names_out())
```

```
# Display TF-IDF matrix
```

```
print(tfidf_matrix.toarray())
```

Neural Approach – Word2Vec

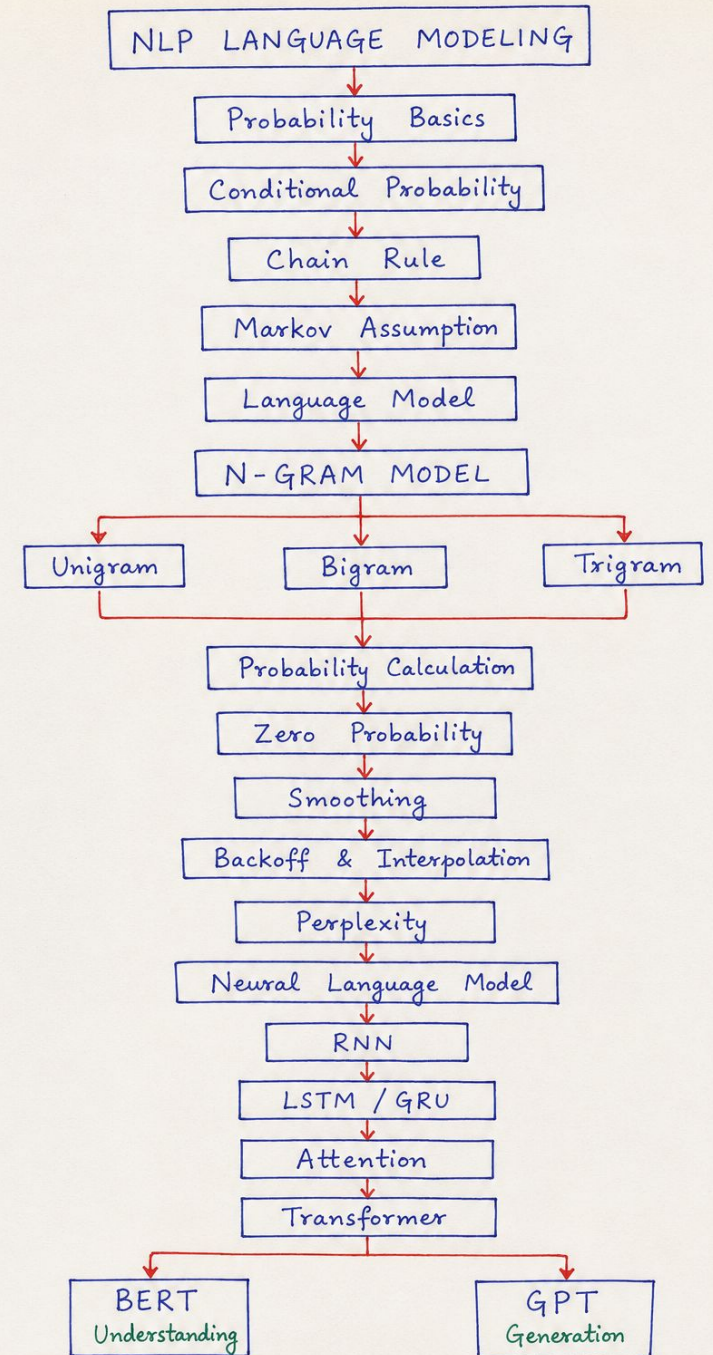
Note : Study in UNIT-3

Word2Vec is a neural network-based technique used to create word embeddings.

- It represents each word as a vector of numbers.
- Words with similar meanings or used in similar contexts get similar vectors.
- Its main goal is to **capture the semantic relationships between words.**
- Word2Vec has two main methods:
 1. **CBOW (Continuous Bag of Words)** – predicts a word from its surrounding words.
 2. **Skip-gram** – predicts surrounding words from a given word.

Probabilistic Language Modeling

Before moving further, a flow needs to be understood for this.



What Is a Language Model?

A Language Model (LM)

learns the relationships between words and sentences.

It predicts **the next suitable word** based on the words that come before it.

EXAMPLE

Mother said grandma was ill,
so last Sunday, Jane and I
visited _____.

her

The model predicts “**her**” because
it understands grandma is a
female person.

Applications of Language Models

Auto-Suggestions

WhatsApp and Gmail suggest the next word or phrase while you type.

Conversational AI

Voice assistants and chatbots like Siri, Alexa, and Google Assistant.

Content Creation

Tools like ChatGPT and Gemini generate stories, poems, articles, and reports.

Sentiment Analysis

Identifies the emotion or opinion behind a piece of text.

Text Summarization

Creates a short summary of a long document, keeping key information.

Transcription

Converts speech or audio into written text, e.g. YouTube captions.

Basic Probability Concepts for Language Models

Concept	Meaning	Example
Probability	Probability of an event happening	$P(\text{rain})$
Conditional Probability	Probability of an event given another event	$P(\text{rain} \mid \text{clouds})$
Joint Probability	Probability of two events happening together	$P(\text{rain}, \text{clouds})$
Chain Rule	Breaks a joint probability into conditional probabilities	$P(A,B,C) = P(A) \times P(B A) \times P(C A,B)$
Independence	Two events do not depend on each other	$P(A,B) = P(A) \times P(B)$

Conditional Probability: The Key Idea

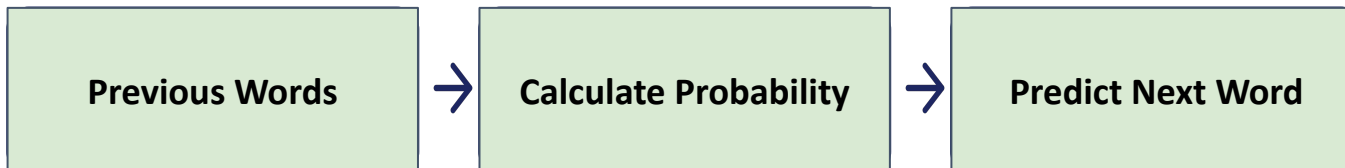
Conditional probability matters most in language modeling: a model predicts a word **based on the words that came before it**.

EXAMPLE “I love machine _____”

$P(\text{learning} \mid \text{I love machine})$

“What is the probability of the word learning, given the previous words I love machine?”

The model compares probabilities of possible next words and selects the most likely one.



WHICH WORD FITS?

I love machine learning.

High probability in this context

I love machine banana.

Very low probability in this context

“learning” has a much higher probability than “banana” given this context.

A Language Model (LM)

- It is a statistical or AI model that learns how words are used together.
- It calculates the probability of a word appearing based on the previous words or context.

Simple Formula: $P(w_n | w_1, w_2, \dots, w_{n-1})$

It means:

What is the probability of the next word w_n , given the words that came before it?

Example

I want to drink _____

The model may predict:

- water → high probability
- juice → high probability
- computer → very low probability

Understand what a language model does.

(A language model can calculate the probability of an entire sentence.)

$$P(w_1, w_2, w_3, \dots, w_n)$$

For example:

I love NLP

The model wants to calculate:

$$P(I, \text{love}, \text{NLP})$$

Using the Chain Rule:

Chain Rule

Breaks a joint probability into conditional probabilities

$$P(A, B, C) = P(A) \times P(B|A) \times P(C|A, B)$$

$$P(I, \text{love}, \text{NLP}) = P(I) \times P(\text{love} | I) \times P(\text{NLP} | I, \text{love})$$

This is the mathematical foundation of **probabilistic language modeling**.

Understanding the Chain Rule in Language Modeling

How a model turns one long sentence into a chain of simple, next-word predictions

$$P(w_1, w_2, \dots, w_n) = \prod P(w_i \mid w_1, \dots, w_{i-1})$$

The Chain Rule of Probability

Any joint probability can be broken into a product of conditional probabilities, one per word, each conditioned on everything before it.

$$P(w_1, w_2, \dots, w_n) = P(w_1) \times P(w_2|w_1) \times P(w_3|w_1, w_2) \times \dots \times P(w_n|w_1, \dots, w_{n-1})$$

1

Sequential

Words are generated one at a time, left to right.

2

Conditional

Every new word depends on all the words that came before it.

3

Multiplicative

Multiply each step's probability to get the full sentence probability.

The Chain Rule of Probability

Example: "I love AI"

Suppose our sentence is:

I → love → AI

The Chain Rule is:

$$P(w_1, w_2, w_3) = P(w_1) \times P(w_2|w_1) \times P(w_3|w_1, w_2)$$

So:

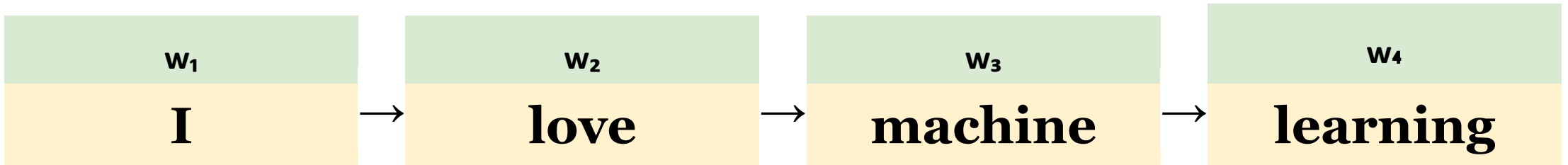
$$P(I, love, AI) = P(I) \times P(love|I) \times P(AI|I, love)$$

Assume:

Step	Probability	Value
1. Sequential	$P(I)$	0.10
2. Conditional	$P(love I)$	0.20
3. Conditional	$P(AI I, love)$	0.50

Each Word, Predicted From What Came Before

Take the sentence "I love machine learning." Each word becomes one link in the chain:



$P(\text{I, love, machine, learning})$: Probability of words in a sentence

$$= P(\text{I}) \times P(\text{love} \mid \text{I}) \times P(\text{machine} \mid \text{I, love}) \times P(\text{learning} \mid \text{I, love, machine})$$

FROM DATA TO PROBABILITIES

Estimating Probabilities from Training Sentences

Turning raw word counts from 5 example sentences into the Chain Rule probabilities

Probability = count of word in context ÷ total possible next words

Five Example Training Sentences

S1 I love machine learning

S2 I love Python

S3 I like machine learning

S4 I like Python

S5 I am learning Python

Probability of the First Word: $P(I)$

Count how many sentences begin with “I,” out of all sentences.

I love machine learning

I love Python

I like machine learning

I like Python

I am learning Python

$P(I)$

sentences starting with “I”

$$= \frac{5}{5}$$

1.0

Every sentence in the corpus starts with “I” — so $P(I)$ is certain.

$$P(I) = \frac{\text{Number of sentences starting with I}}{\text{Total sentences}}$$

Probability of “love” After “I”: P(love | I)

Look at every word that follows “I” across all sentences, and count how often each one appears.

- I love machine learning
- I love Python
- I like machine learning
- I like Python
- I am learning Python

After “I”	Count
love	2
like	2
am	1
Total	5

P(love | I)

$$= \frac{2}{5}$$

0.40

2 out of the 5 sentences that start with “I” continue with “love.”

Probability of “machine” After “I love”

$P(\text{machine} \mid \text{I, love})$ — narrow the corpus to sentences beginning “I love,” then see what comes next.

I love machine learning

I love Python

Next word	Count
machine	1
Python	1
Total	2

$P(\text{machine} \mid \text{I, love})$

$$= \frac{1}{2}$$

0.50

Of the 2 sentences beginning “I love,” exactly 1 continues with “machine.”

Probability of “learning” After “I love machine”

Only one sentence in the whole corpus contains this exact three-word prefix.

I love machine → learning

Next word

Count

learning

1

Total

1

$P(\text{learning} \mid \text{I, love, machine})$

$$= \frac{1}{1}$$

1.0

The only sentence with this prefix continues with “learning” — so the probability is certain.

Combine the Steps for “I love machine learning”

$$\begin{array}{ccccccc} \boxed{1.0} & \times & \boxed{0.40} & \times & \boxed{0.50} & \times & \boxed{1.0} = \boxed{0.20} \\ P(I) & & P(\text{love}|I) & & P(\text{machine}|I,\text{love}) & & P(\text{learning}|\dots) \end{array}$$

$$P(I, \text{love}, \text{machine}, \text{learning}) = 1.0 \times 0.40 \times 0.50 \times 1.0 = 0.20$$

Question 1:

Consider the following corpus of 10 sentences.

1. The smart student learns Python.
2. The smart student reads books.
3. The smart teacher learns Python.
4. The good student learns Python.
5. The good teacher reads books.
7. A smart student learns Python.
8. The smart student reads Java.
9. A good student learns Java.
10. The smart teacher reads books.

Using a 4-gram (Chain Rule) language model, calculate the probability of the sentence:

"The smart student learns Python."

That is, compute:

$$P(\text{The, smart, student, learns, Python})$$

using the Chain Rule:

$$\begin{aligned} &P(\text{The}) \times P(\text{smart} | \text{The}) \times P(\text{student} | \text{The, smart}) \times \\ &\quad P(\text{learns} | \text{The, smart, student}) \times \\ &\quad P(\text{Python} | \text{The, smart, student, learns}) \end{aligned}$$

Solution:

Step 1: Count sentences starting with "The".

"The" starts 8 sentences out of 10.

$$P(\text{The}) = \frac{8}{10} = 0.8$$

Step 2: Compute conditional probabilities.

Probability	Count (Numerator)	Count (Denominator)	Value
$P(\text{smart} \text{The})$	"The smart" occurs 6 times (S: 1, 2, 3, 5, 8, 10)	"The" occurs 8 times	$\frac{6}{8} = 0.75$
$P(\text{student} \text{The, smart})$	"The smart student" occurs 4 times (S: 1, 2, 5, 8)	"The smart" occurs 6 times	$\frac{4}{6} = 0.667$
$P(\text{learns} \text{The, smart, student})$	"The smart student learns" occurs 2 times (S: 1, 5)	"The smart student" occurs 4 times	$\frac{2}{4} = 0.5$
$P(\text{Python} \text{The, smart, student, learns})$	"The smart student learns Python" occurs 1 time (S: 1)	"The smart student learns" occurs 2 times (S: 1, 5)	$\frac{1}{2} = 0.5$

Step 3: Apply the Chain Rule.

$$P(\text{The, smart, student, learns, Python})$$

$$= 0.8 \times 0.75 \times 0.667 \times 0.5 \times 0.5$$

$$= \frac{8}{10} \times \frac{6}{8} \times \frac{4}{6} \times \frac{2}{4} \times \frac{1}{2}$$

$$= \frac{1}{10} = \boxed{0.1}$$

Therefore,

$$P(\text{The, smart, student, learns, Python}) = 0.1 \text{ (or } \underline{\underline{10\%}}).$$

Types of Language Models

Probabilistic Language Models

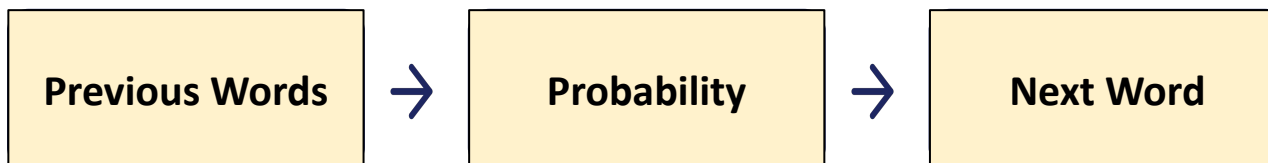
Predict the next word by calculating probabilities — commonly using **N-grams**.

EXAMPLE

“I am going to ____”

The model calculates probabilities for college, school, market, etc. and selects the most likely word.

MAIN IDEA



1 · Probabilistic

2 · Neural Network

This we study in unit 4

LIMITATIONS

- Consider only a limited number of previous words
- Cannot understand long-term context well
- As N increases, the number of possible N-Grams grows very large
- Many N-Grams never appear in training data — the sparsity problem
- Large N-Gram models require more storage and computation

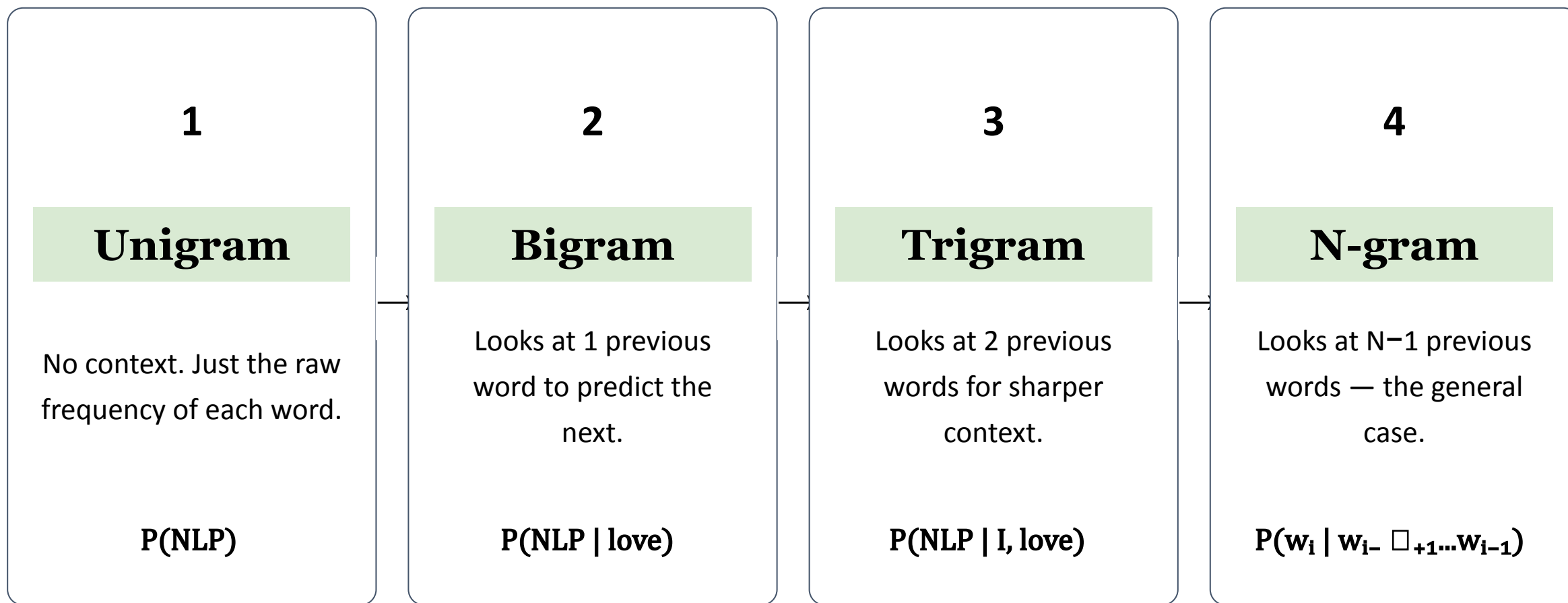
N-Gram Language Model

From counting single words to predicting whole sentences — unigram, bigram, and trigram models built from one tiny corpus

$$P(w_i | w_{i-1}) = \text{Count}(w_{i-1}, w_i) / \text{Count}(w_{i-1})$$

From Unigram to N-Gram

Each step adds one more word of context — and one more layer of predictive power.



Training Corpus with Sentence Boundaries

S1	⟨s⟩	I	love	NLP	⟨/s⟩
S2	⟨s⟩	I	love	Python	⟨/s⟩
S3	⟨s⟩	I	like	NLP	⟨/s⟩
S4	⟨s⟩	You	love	NLP	⟨/s⟩
S5	⟨s⟩	You	like	Python	⟨/s⟩

⟨s⟩ = beginning of sentence ⟨/s⟩ = end of sentence

Unigram: Counting Individual Words

The simplest model — no context at all, just how often each word appears in the corpus.

S1	<s>	I	love	NLP	</s>
S2	<s>	I	love	Python	</s>
S3	<s>	I	like	NLP	</s>
S4	<s>	You	love	NLP	</s>
S5	<s>	You	like	Python	</s>

Word	Count
I	3
love	3
NLP	3
Python	2
like	2
You	2
Total	15

$P(w) = \text{Count}(w) / \text{Total Words}$

$P(I)$	$= 3/15 =$	0.20
$P(\text{love})$	$= 3/15 =$	0.20
$P(NLP)$	$= 3/15 =$	0.20
$P(\text{Python})$	$= 2/15 =$	0.133

A unigram simply reflects raw word frequency — it ignores word order entirely.

Bigram: Using the Previous Word

A bigram looks at two consecutive words and asks: given the word before, what comes next?

$$P(w_i | w_{i-1}) = \text{Count}(w_{i-1}, w_i) / \text{Count}(w_{i-1})$$

Probability = (number of times the two words occur together) ÷ (number of times the first word occurs)

I love

love NLP

I like

like NLP

Breaking Each Sentence into Bigrams

Slide a two-word window across every sentence to generate its bigrams.

<s> I love NLP </s>

<s> I

I love

love NLP

NLP </s>

<s> I love Python </s>

<s> I

I love

love Python

Python </s>

<s> I like NLP </s>

<s> I

I like

like NLP

NLP </s>

<s> You love NLP </s>

<s> You

You love

love NLP

NLP </s>

<s> You like Python </s>

<s> You

You like

like Python

Python </s>

Counting the Bigrams

Bigram	Count
⟨s⟩ I	3
⟨s⟩ You	2
I love	2
I like	1
love NLP	2
love Python	1

Bigram	Count
like NLP	1
like Python	1
You love	1
You like	1
NLP ⟨/s⟩	3
Python ⟨/s⟩	2

CONDITIONAL PROBABILITY

What Follows “I”? love or like

Count(I) = 3 across the corpus. Now split that 3 between its two possible continuations.

<s> I	I love	love NLP	NLP </s>
<s> I	I love	love Python	Python </s>
<s> I	I like	like NLP	NLP </s>
<s> You	You love	love NLP	NLP </s>
<s> You	You like	like Python	Python </s>

$$P(\text{love} \mid I)$$

$$= \frac{2}{3}$$

0.667

$$\text{Count}(I, \text{love}) = 2 \quad \text{Count}(I) = 3$$

$$P(\text{like} \mid I)$$

$$= \frac{1}{3}$$

0.333

$$\text{Count}(I, \text{like}) = 1 \quad \text{Count}(I) = 3$$

$$P(\text{love} \mid I) + P(\text{like} \mid I) = 2/3 + 1/3 = 1.0$$

CONDITIONAL PROBABILITY

What Follows “love”? NLP or Python

Count(love) = 3 across the corpus. Split between its two possible continuations.

<s> I	I love	love NLP	NLP </s>
<s> I	I love	love Python	Python </s>
<s> I	I like	like NLP	NLP </s>
<s> You	You love	love NLP	NLP </s>
<s> You	You like	like Python	Python </s>

$$P(\text{NLP} \mid \text{love})$$

$$= \frac{2}{3}$$

0.667

$$\text{Count}(\text{love}, \text{NLP}) = 2 \quad \text{Count}(\text{love}) = 3$$

$$P(\text{Python} \mid \text{love})$$

$$= \frac{1}{3}$$

0.333

$$\text{Count}(\text{love}, \text{Python}) = 1 \quad \text{Count}(\text{love}) = 3$$

$$P(\text{NLP} \mid \text{love}) + P(\text{Python} \mid \text{love}) = 2/3 + 1/3 = 1.0$$

Predicting the Next Word

At each step, the model picks the continuation with the higher probability.

Input:

I

love

0.667

like

0.333

Prediction: love

Input:

I love

NLP

0.667

Python

0.333

Prediction: NLP

Multiplying the Chain: “I love NLP”

Chain the conditional probabilities together to score a whole sentence.

$$P(\text{I, love, NLP}) = P(\text{I}) \times P(\text{love} \mid \text{I}) \times P(\text{NLP} \mid \text{love}) = 0.20 \times 0.667 \times 0.667$$

$$0.20 \times 0.667 \times 0.667 \approx 8.89\%$$

$$\approx \mathbf{0.0889}$$

The Zero-Frequency Problem

What happens when a model meets a word pair it has never seen?

Try scoring: "I love Java"

$$\text{Count}(\text{love}, \text{Java}) = 0 \rightarrow P(\text{Java} \mid \text{love}) = 0/3 = 0$$

$$P(I) \times P(\text{love} \mid I) \times P(\text{Java} \mid \text{love}) = P(I) \times P(\text{love} \mid I) \times 0 = 0$$

One missing bigram zeroes out the probability of the entire sentence — even if every other word pair was common.

Fixed with smoothing techniques:

Add-One / Laplace

Add-k Smoothing

Good-Turing

Backoff

Interpolation

Trigram: Adding One More Word of Context

A trigram conditions on the two previous words instead of just one.

$$P(w_i | w_{i-2}, w_{i-1}) = \text{Count}(w_{i-2}, w_{i-1}, w_i) / \text{Count}(w_{i-2}, w_{i-1})$$

$$P(\text{NLP} | \text{I}, \text{love})$$

$$= \frac{1}{2}$$

0.5

$$\text{Count}(\text{I}, \text{love}, \text{NLP}) = 1 \quad \text{Count}(\text{I}, \text{love}) = 2$$

I

love

NLP

I

love

Python

Bigram vs Trigram vs N-Gram

Model	Looks At	Example
Unigram	No previous word	$P(\text{NLP})$
Bigram	1 previous word	$P(\text{NLP} \mid \text{love})$
Trigram	2 previous words	$P(\text{NLP} \mid \text{I, love})$
4-gram	3 previous words	$P(w_i \mid w_{i-3}, w_{i-2}, w_{i-1})$
N-gram	$N-1$ previous words	$P(w_i \mid w_{i-n+1}, \dots, w_{i-1})$

The core idea: **probability = Count(context, word) ÷ Count(context)**...

More context means sharper predictions, but also more data needed
and a higher chance of hitting zero counts.

$$P(w_i \mid w_{i-1}) = C(w_{i-1}, w_i) / C(w_{i-1})$$

Estimating the Probability of a Word

Question : Consider the following training corpus:

<s> I like data science </s>

<s> I like machine learning </s>

<s> I study data science </s>

Using the Bigram Maximum Likelihood Estimation (MLE) formula:

$$P(w_i | w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

Calculate the probability of the sentence:

$$P(\text{"I like data science"})$$

Show the bigram counts, individual probabilities, and final sentence probability.

Estimating the Probability of a Word

Solution: Bigram Probability using MLE

Given Training Corpus

1. `<s> I like data science </s>`
2. `<s> I like machine learning </s>`
3. `<s> I study data science </s>`

Step 1: Count the required words

For the sentence "I like data science", we need:

Bigram	Bigram Count	Previous Word Count	Probability
<code><s> → I</code>	3	3	$3/3 = 1.00$
<code>I → like</code>	2	3	$2/3 = 0.67$
<code>like → data</code>	1	2	$1/2 = 0.50$
<code>data → science</code>	2	2	$2/2 = 1.00$
<code>science → </s></code>	2	2	$2/2 = 1.00$

Estimating the Probability of a Word

Step 2: Calculate Sentence Probability

The probability of the sentence is:

$$\begin{aligned} &P(\text{I like data science}) \\ &= P(I | < s >) \times P(\text{like} | I) \times P(\text{data} | \text{like}) \times P(\text{science} | \text{data}) \times P(< /s > | \text{science}) \end{aligned}$$

Substitute the values:

$$\begin{aligned} &= 1.00 \times 0.67 \times 0.50 \times 1.00 \times 1.00 \\ &= 0.335 \end{aligned}$$

Smoothing in N-Gram Language Models

Fixing the zero-probability problem so unseen words still get a chance

Why Do We Need Smoothing?

IN THE CORPUS

I love NLP

NOT IN THE CORPUS

I love AI

Using Maximum Likelihood Estimation:

$$P(\text{AI} \mid \text{love}) = C(\text{love}, \text{AI}) / C(\text{love})$$

Since “love AI” never appeared: $C(\text{love}, \text{AI}) = 0$

$$P(\text{AI} \mid \text{love}) = 0$$

→ the Zero Probability Problem

What Is Smoothing?

Smoothing adjusts probability estimates so unseen n-grams get a small, non-zero probability instead of zero.

INSTEAD OF

$$P(\text{AI} \mid \text{love}) = 0$$



WE WANT

$$P(\text{AI} \mid \text{love}) > 0$$

Add-One / Laplace Smoothing

The simplest smoothing technique: add 1 to every n-gram count.

$$P(w_i | w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + V}$$

Where:

$C(w_{i-1}, w_i)$	bigram count — how often this exact word pair occurred
$C(w_{i-1})$	previous-word count — how often the first word occurred
V	vocabulary size — the number of distinct words the model knows

Example: $P(\text{AI} \mid \text{love})$

$C(\text{love}, \text{AI})$

0

$C(\text{love})$

1

V

4

$$\begin{aligned} P(\text{AI} \mid \text{love}) &= (C(\text{love}, \text{AI}) + 1) / (C(\text{love}) + V) \\ &= (0 + 1) / (1 + 4) = 1 / 5 \end{aligned} \qquad \qquad \qquad = \mathbf{0.20}$$

Before smoothing: $P(\text{AI} \mid \text{love}) = 0$

After smoothing: $P(\text{AI} \mid \text{love}) = 0.20$

The Markov Assumption

“ The next word depends only on a limited number of previous words not on the entire history. ”

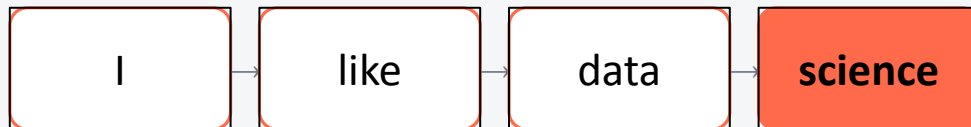
Predicting “science” in “I like data science”

The same word, predicted two very different ways.

WITHOUT MARKOV ASSUMPTION

Consider the complete history

$$P(\text{science} \mid \text{I, like, data})$$



all previous words remembered

The model remembers every word that came before — the full sentence history.

WITH BIGRAM MARKOV ASSUMPTION

Consider only the immediately previous word

$$P(\text{science} \mid \text{data})$$



only 1 word remembered

The model forgets “I” and “like” entirely — it only remembers “data.”

Markov Assumption vs Bigram Model

	Markov Assumption	Bigram Model
What it is	A principle about how much past information is needed	A specific N-gram language model
Claim	Next word depends only on a limited previous context	Next word depends on exactly 1 previous word
Flexibility	Can be applied with different context sizes	Always uses exactly one previous word
Scope	General idea	Specific implementation of that idea
Example	$P(w_i w_{i-1})$ or $P(w_i w_{i-2},w_{i-1})$	$P(w_i w_{i-1})$

NAIVE BAYES

A Probability-Based Approach to Text Classification

Spam Detection • Sentiment Analysis • Document & News Classification •
Language Detection

What Is Naive Bayes?

Naive Bayes is a supervised machine learning classification algorithm based on Bayes' Theorem.

It is mainly used to classify data into different categories.

Common NLP use cases

- Spam detection
- Sentiment analysis
- Text / document classification
- News classification
- Language classification

“This movie is excellent and amazing.”

Naive Bayes estimates:

Positive → 90%

Negative → 10%

Result: classified as POSITIVE

Why the Name “Naive Bayes”?

Naive = simple assumption without considering all the complexity.

B

Bayes

Based on Bayes’ Theorem — a rule for calculating probabilities from known evidence.

N

Naive

Assumes every word is independent of every other word when calculating probability.

`“I love this movie”`

treated as 4 separate, unrelated features — not realistic, but it makes the math fast and simple.

Bayes' Theorem

$$P(C | X) = \frac{P(X | C) \times P(C)}{P(X)}$$

Posterior = (Likelihood × Prior) / Evidence

C

Class

X

Input text

P (C | X)

Posterior

P (X | C)

Likelihood

P (C)

Prior

P (X)

Evidence

Meaning of Each Term

Prior

What we believed before seeing the new evidence

Likelihood

How likely the evidence is, if our assumption is true

Evidence

How likely the evidence is overall

Posterior

What we believe after seeing the evidence

Example

Is an email spam?

Step 1

Prior

20% of emails are spam

Step 2

Likelihood

“Win money! Free prize!” is
common in spam

Step 3

Posterior

Updated probability the
email is spam

Step 1 — The Prior

What we believed before looking at this specific email

Before checking the email, we already know:

20% of all emails are spam.

This is our starting belief — based on general history, not this particular message.

Prior

$P(\text{Spam})$

= 0.20

20 out of every 100 emails are spam, on average.

Step 2 — The Likelihood

How well the new evidence fits our assumption

The email contains: “Win money! Free prize!”

Words like these are very common in spam emails — so if this email really is spam, seeing these words is quite likely.

Likelihood

$P(\text{“Win money! Free prize!”} \mid \text{Spam})$

= 0.80

Step 3 — Evidence & Posterior

Evidence — the overall probability of seeing these words in any email, spam or not.

$$\text{Posterior} = \frac{\text{Likelihood} \times \text{Prior}}{\text{Evidence}}$$

The Posterior is the probability the email is spam, AFTER factoring in those spammy-sounding words — combining what we believed before (Prior) with how well the evidence fits (Likelihood).

The Naive Assumption, Applied

Every word is treated as an independent feature

“movie is amazing”

movie

is

amazing

→ treated as 3 separate, unrelated features

$$\begin{aligned} P(\text{movie}, \text{amazing}, \text{is} \mid \text{Positive}) &\approx \\ P(\text{movie} \mid \text{Positive}) &\times P(\text{amazing} \mid \text{Positive}) \times P(\text{is} \mid \text{Positive}) \end{aligned}$$

We repeat the same calculation for the Negative class, then compare the two scores.

Example: The Training Data

Review	Class
"I love this movie"	Positive
"This movie is amazing"	Positive
"I hate this movie"	Negative
"This movie is boring"	Negative
Task: classify a new sentence → "I love this movie"	

Step 1 — Prior Probabilities

$P(C)$: how common is each class, before reading any words?

4 total training documents — 2 Positive, 2 Negative

$P(\text{Positive})$

$$\frac{2}{4} = 0.5$$



Class
Positive
Positive
Negative
Negative

$P(\text{Negative})$

$$\frac{2}{4} = 0.5$$

These are the baseline odds — before any word evidence is applied.

Step 2 — Word Probabilities

$P(\text{word} \mid \text{class})$: how often each word appears within each class **$P(X/C)$**

Positive reviews: “I love this movie” + “This movie is amazing” — 8 words total

Negative reviews: “I hate this movie” + “This movie is boring” — 8 words total

Word	Count in Positive	$P(\text{word} \mid \text{Positive})$	Count in Negative	$P(\text{word} \mid \text{Negative})$
I	1	1 / 8	1	1 / 8
love	1	1 / 8	0	0 / 8
this	2	2 / 8	2	2 / 8
movie	2	2 / 8	2	2 / 8

Watch “love” in the Negative column — it never occurs there, so its probability is 0.

Positive reviews

Multiply these probabilities:

$$\begin{aligned}P(\textit{Review}|\textit{Positive}) &= \frac{1}{8} \times \frac{1}{8} \times \frac{2}{8} \times \frac{2}{8} \\ &= \frac{4}{4096} = \frac{1}{1024}\end{aligned}$$

Now multiply by the prior:

$$\begin{aligned}P(\textit{Positive}) \times P(\textit{Review}|\textit{Positive}) \\ &= \frac{1}{2} \times \frac{1}{1024} = \frac{1}{2048}\end{aligned}$$

Negative reviews:

Therefore:

$$\begin{aligned} P(\textit{Review}|\textit{Negative}) &= \frac{1}{8} \times 0 \times \frac{2}{8} \times \frac{2}{8} \\ &= 0 \end{aligned}$$

And:

$$P(\textit{Negative}) \times P(\textit{Review}|\textit{Negative}) = \frac{1}{2} \times 0 = 0$$

What is $P(X)$?

$X = \text{"I love this movie"}$

$P(X)$ means: $(X = \text{Review})$

The overall probability of observing the input X , considering all possible classes.

If there are two classes, for example Positive and Negative, then:

$$P(X) = P(X \mid \text{Positive})P(\text{Positive}) + P(X \mid \text{Negative})P(\text{Negative})$$

Calculate $P(X)$

Since there are two classes:

$$P(X) = [P(X|Positive) \times P(Positive)] + [P(X|Negative) \times P(Negative)]$$

Substitute the values:

$$P(X) = \frac{1}{2048} + 0$$

Therefore:

$$P(X) = \frac{1}{2048}$$

Now calculate the Posterior

For Positive:

$$P(Positive|X) = \frac{P(X|Positive) \times P(Positive)}{P(X)}$$

$$= \frac{\frac{1}{2048}}{\frac{1}{2048}}$$

$$P(Positive|X) = 1$$

or

$$100\%$$

For Negative:

$$P(Negative|X) = \frac{0}{\frac{1}{2048}}$$

$$P(Negative|X) = 0$$

Therefore:

“I love this movie” → Positive

The Zero Probability Problem

One missing word can wipe out an entire class

Because

$$P(\text{love} \mid \text{Negative}) = 0$$

...the entire product becomes:

0

Naive Bayes would eliminate the Negative class entirely — just because one word, “love,” happened not to appear in the Negative training data.

The fix: Laplace Smoothing → next slide

The Fix — Laplace Smoothing

Add 1 to every word count so no probability is ever truly zero

$$P(\text{word} \mid \text{class}) = \frac{\text{Count}(\text{word}, \text{class}) + 1}{\text{Total Words in Class} + V}$$

$V = \text{vocabulary size}$

Vocabulary size (V) = 8 • Total words per class = 8 • Smoothed denominator (pos+Neg)= 8 + 8 = 16

Word	Smoothed P(word Positive)	Smoothed P(word Negative)
I	(1+1) / 16 = 2 / 16	(1+1) / 16 = 2 / 16
love	(1+1) / 16 = 2 / 16	(0+1) / 16 = 1 / 16
this	(2+1) / 16 = 3 / 16	(2+1) / 16 = 3 / 16
movie	(2+1) / 16 = 3 / 16	(2+1) / 16 = 3 / 16

“love” now has a real, non-zero probability in the Negative class.

Final Scores & Classification

Multiplying it all together for “I love this movie”

Score(Positive)

$$0.5 \times 2/16 \times 2/16 \times 3/16 \times 3/16$$

$$\approx \mathbf{0.0000824}$$

Score(Negative)

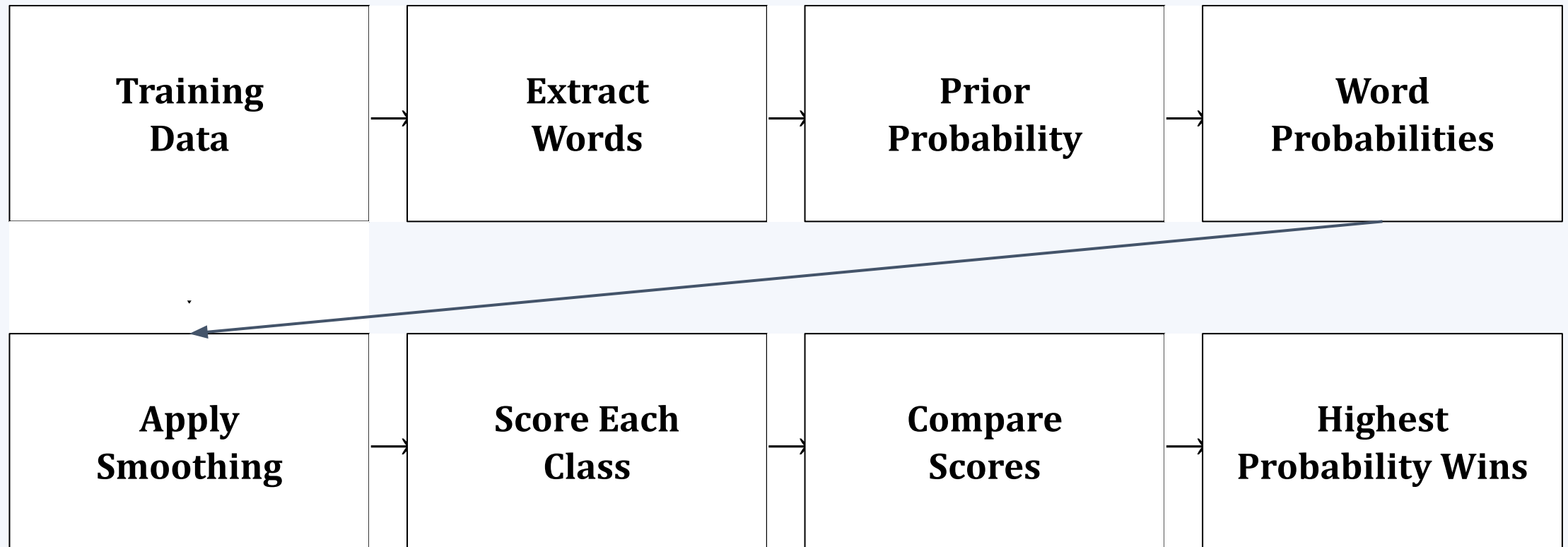
$$0.5 \times 2/16 \times 1/16 \times 3/16 \times 3/16$$

$$\approx \mathbf{0.0000412}$$

Score(Positive) > Score(Negative) → Final Prediction: POSITIVE

The Full Naive Bayes Workflow

Every step we just walked through, end to end



Choose the class with the highest final probability — that's the prediction.

Strengths & Limitations

Strengths

- Simple to understand
- Fast to train and predict
- Works well with text data
- Handles high-dimensional data well
- Needs relatively little training data
- Great for spam & sentiment tasks

Limitations

- Independence assumption is often unrealistic
- Doesn't capture word order
- Doesn't truly understand context
- Zero probabilities without smoothing
- Can lag behind modern deep-learning models

Part 1: Problem Statement

A company wants to classify incoming emails as Spam or Not Spam based on a training dataset of previously labeled emails.

Training Dataset

Email ID	Email Content	Category (Class)
1	Win <u>free prize</u>	Spam
2	Win free <u>money</u>	Spam
3	Meeting at <u>office</u>	Not Spam
4	Meeting at school	Not Spam

New Email to Classify (\$X\$)

X = "Win free prize"

Required Calculations

1. Calculate prior probabilities:
 $P(\text{Spam})$ and $P(\text{NotSpam})$
2. Calculate the probability of each word given Spam
3. Calculate the probability of each word given NotSpam
4. Calculate likelihoods: $P(X \mid \text{Spam})$ and $P(X \mid \text{NotSpam})$
5. Calculate evidence probability:
 $P(X)$
6. Calculate posterior probabilities:
 $P(\text{Spam} \mid X)$ and $P(\text{NotSpam} \mid X)$
7. Determine final classification for the new email

Step 1: Calculate Prior Probabilities

Total number of training emails = 4

- $P(\text{Spam}) = (\text{Number of Spam emails}) / (\text{Total emails}) = 2 / 4 = \mathbf{0.5}$
- $P(\text{NotSpam})$ = (Number of Not Spam emails) / (Total emails) = 2 / 4 = **0.5**

Step 2: Word Frequency Counts

Spam Class: Total word count = 6 words

Word	Frequency in Spam
Win	2
free	2
prize	1
money	1

Not Spam Class: Total word count = 6 words

Word	Frequency in Not Spam
Meeting	2
at	2
office	1
school	1

Step 3: Conditional Probabilities of Words Given Class

Probability of each word given Spam:

- $P(\text{"Win"} \mid \text{Spam}) = 2 / 6 = 1/3 \approx \mathbf{0.333}$
- $P(\text{"free"} \mid \text{Spam}) = 2 / 6 = 1/3 \approx \mathbf{0.333}$
- $P(\text{"prize"} \mid \text{Spam}) = 1 / 6 \approx \mathbf{0.167}$

Probability of each word given Not Spam:

- $P(\text{"Win"} \mid \text{NotSpam}) = 0 / 6 = \mathbf{0.000}$
- $P(\text{"free"} \mid \text{NotSpam}) = 0 / 6 = \mathbf{0.000}$
- $P(\text{"prize"} \mid \text{NotSpam}) = 0 / 6 = \mathbf{0.000}$

New Email to Classify

`X = "Win free prize"`

Win <u>free</u> prize	Spam
Win free <u>money</u>	Spam
Meeting at <u>office</u>	Not Spam
Meeting at school	Not Spam

Step 4: Calculate Likelihoods $P(X \mid \text{Class})$

For the test email $X = \{\text{"Win", "free", "prize"}\}$:

$$\begin{aligned}P(X \mid \text{Spam}) &= P(\text{"Win"} \mid \text{Spam}) \times P(\text{"free"} \mid \text{Spam}) \times P(\text{"prize"} \mid \text{Spam}) \\&= (1/3) \times (1/3) \times (1/6) \\&= 1 / 54 \\&\approx 0.0185\end{aligned}$$

$$\begin{aligned}P(X \mid \text{NotSpam}) &= P(\text{"Win"} \mid \text{NotSpam}) \times P(\text{"free"} \mid \text{NotSpam}) \times P(\text{"prize"} \mid \text{NotSpam}) \\&= 0 \times 0 \times 0 \\&= 0\end{aligned}$$

Step 5: Calculate Total Probability $P(X)$

Using the Law of Total Probability:

$$\begin{aligned}P(X) &= [P(X \mid \text{Spam}) \times P(\text{Spam})] + [P(X \mid \text{NotSpam}) \times P(\text{NotSpam})] \\&= (1/54 \times 0.5) + (0 \times 0.5) \\&= 1 / 108 \\&\approx 0.00926\end{aligned}$$

Step 6: Calculate Posterior Probabilities using Bayes' Theorem

$$\begin{aligned}P(\text{Spam} \mid X) &= [P(X \mid \text{Spam}) \times P(\text{Spam})] / P(X) \\&= (1/108) / (1/108)\end{aligned}$$

$$= 1.0 \text{ (100\%)}$$

$$\begin{aligned}P(\text{NotSpam} \mid X) &= [P(X \mid \text{NotSpam}) \times P(\text{NotSpam})] / P(X) \\&= 0 / (1/108) \\&= 0.0 \text{ (0\%)}\end{aligned}$$

Step 7: Final Classification Result

Conclusion:

Since $P(\text{Spam} \mid X) = 1.0$ is greater than $P(\text{NotSpam} \mid X) = 0.0$, the email "Win free prize" is classified as SPAM.

GENERATIVE MODELS OF LANGUAGE

How machines learn to predict, and generate, the next word

N-Gram → **RNN** → **LSTM** → **Transformer** → **GPT**

What Are Generative Models of Language?

Models that learn patterns from text, then use them to generate new text

The core idea:

Predict the next word or token, based on the words that already appeared.

In simple words

They learn how language works, then generate new words, sentences, or paragraphs — one piece at a time.

The Repeating Loop

- 1 Look at the words so far
- 2 Predict the most likely next word
- 3 Add it to the text
- 4 Repeat

This process continues until the required text is generated.

How It Works: One Word at a Time

Model receives:

“I am going to”

Predicts possible next words:

school college market office

Chosen → “I am going to school”

Predict again → “I am going to school today”

...and this process continues until the generated text is complete.

Four Types of Generative Language Models

1

N-Gram

Predicts using a fixed window of previous words

2

RNN / LSTM

Uses memory to carry context across a sequence

3

Transformer

Uses attention to relate any two words directly

4

GPT-Like

A Transformer trained to generate, one token at a time

1. N-Gram Language Models

Traditional statistical models: predict using a fixed number of previous words

1 Bigram — looks at 1 previous word

I → like

like → Python

Python → programming

2 Trigram — looks at 2 previous words

"I like" → Python

Uses the previous two words to predict the next one.

Advantages

- Simple to understand
- Easy to implement
- Requires less computational power

Limitations

- Cannot remember long context
- Needs large storage for word combinations
- Struggles with unseen word combinations

2. RNN & LSTM Language Models

Neural networks that carry memory across a sequence of words

RNN — Recurrent Neural Network

Processes words one by one, keeping a hidden state with information from earlier words.

“I went to the market because I needed some —”

predicts → **food**

Key idea: the RNN has a type of memory.

Advantages

- Handles sequences
- Uses previous context
- LSTM remembers longer sequences

LSTM — Long Short-Term Memory

An improved RNN, built to remember important information for longer and forget less.

“Riya went to the library because she wanted to read a book.”

“**she**” → **correctly linked to Riya**

Limitations

- Slow to train
- Struggles with very long sequences
- Less efficient than Transformers

3. Transformer-Based Language Models

Attention: figuring out which words matter to each other

Instead of processing words strictly one after another like an RNN, Transformers use an attention mechanism to weigh every word against every other word.

“The student went to the college because he had an exam.”

To understand “he”, the model connects it back to “student” — attention finds that link directly, no matter the distance.

Main idea: attention helps the model focus on the important words when understanding or generating text.

Examples of Transformer-based models

BERT

GPT

T5

LLaMA

Not every Transformer is primarily generative — BERT is mainly an understanding/encoding model, while GPT is built for autoregressive generation.

4. GPT-Like Models

A Transformer trained to predict the next token, one at a time

GPT (Generative Pre-trained Transformer) learns from a very large amount of text, and learns to predict the next token.

“Artificial Intelligence is” → changing

“Artificial Intelligence is changing” → the

“Artificial Intelligence is changing the” → ...

...and so on, until GPT produces a complete response.

GPT can generate:

Answers

Stories

Emails

Summaries

Articles

Conversations

Code

Comparison

All five approaches, side by side

Model	Basic Idea	Context
N-Gram	Predict next word using a fixed number of previous words	Short
RNN	Uses recurrent memory to process sequences	Better
LSTM	RNN with improved long-term memory	Longer
Transformer	Uses attention to understand relationships	Long & powerful
GPT	Transformer-based model that generates text token by token	Very powerful

*Thank
you!*